

CHAPTER 18

SOFTWARE TESTING STRATEGIES

STRATEGIC APPROACH TO SOFTWARE TESTING

Generic characteristics of software testing strategies:-

- Testing begins at module level and works outward towards the integration of the entire computer based system.
- Different testing techniques are required at different points in time.
- Testing is conducted by the s/w developer and ITG (Independent Test Group) for large projects.
- Testing and Debugging are different and Debugging is essential in any testing strategy.

Verification and Validation

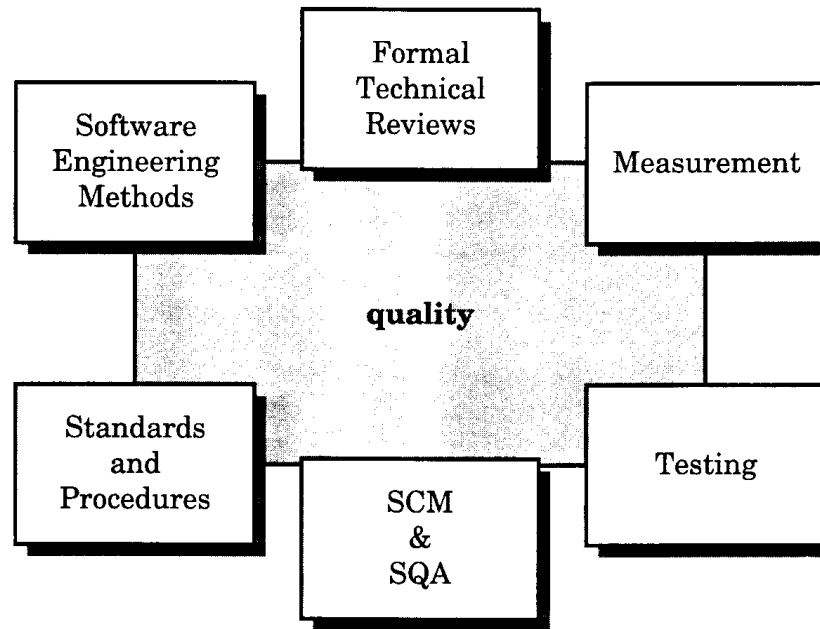


FIGURE 17.1.
Achieving software
quality

Verification -- Does the product meet its specifications? Are we building the product right?

Validation -- Does the product perform as desired? Are we building the right product?

Software Testing Strategy

A Software Testing Strategy

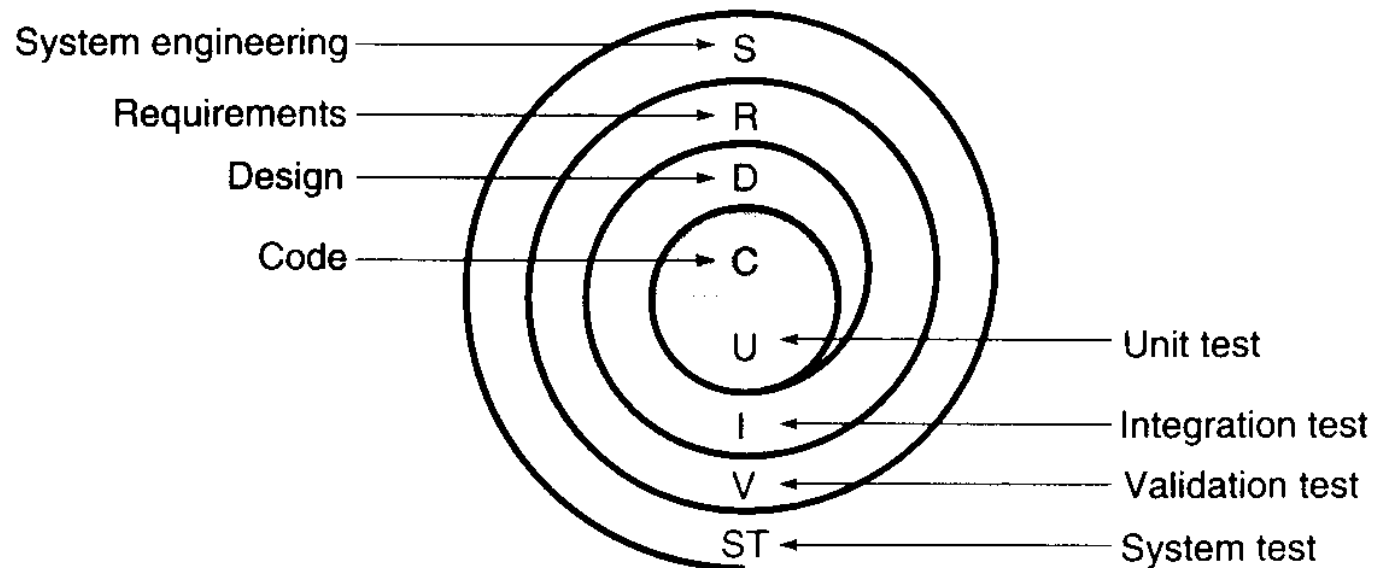


FIGURE 17.2.
Testing strategy

Software Testing Strategy

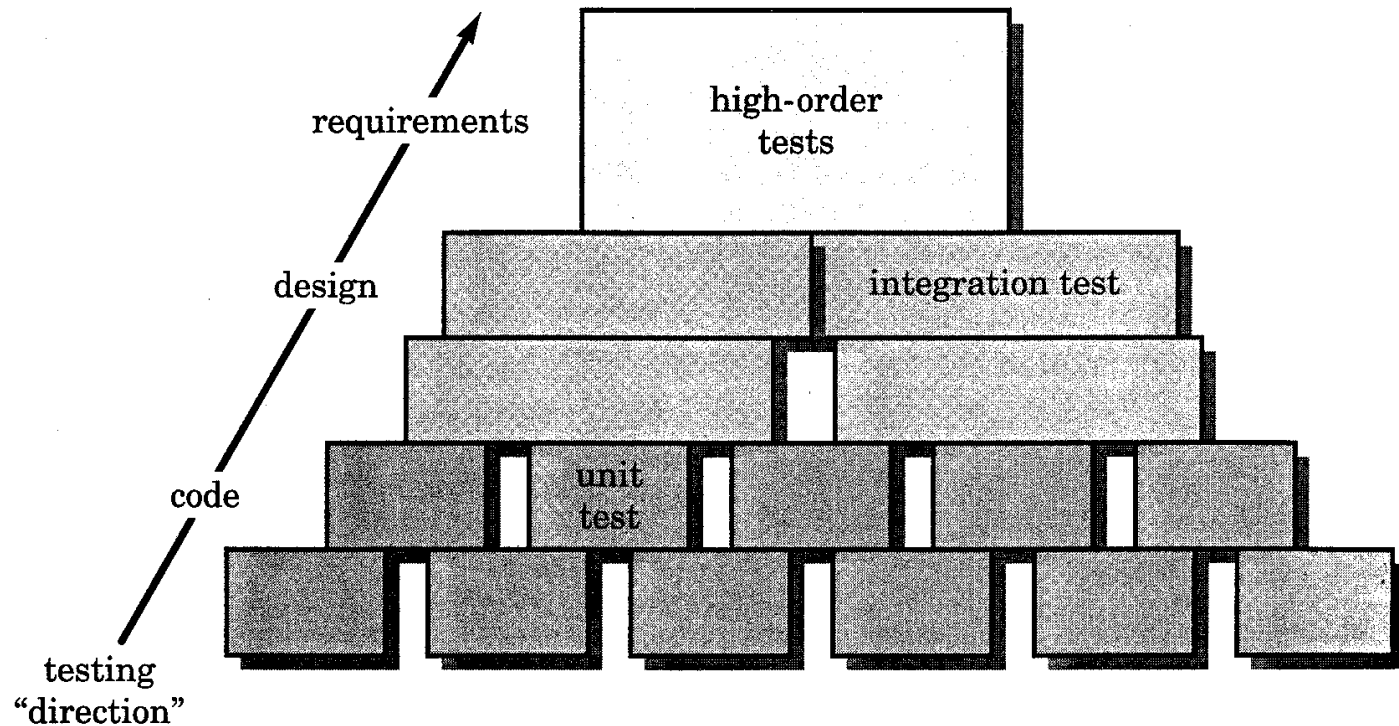


FIGURE 17.3. Software testing steps

UNIT TESTING

Unit testing -- focuses on the smallest element of software design viz. the module.

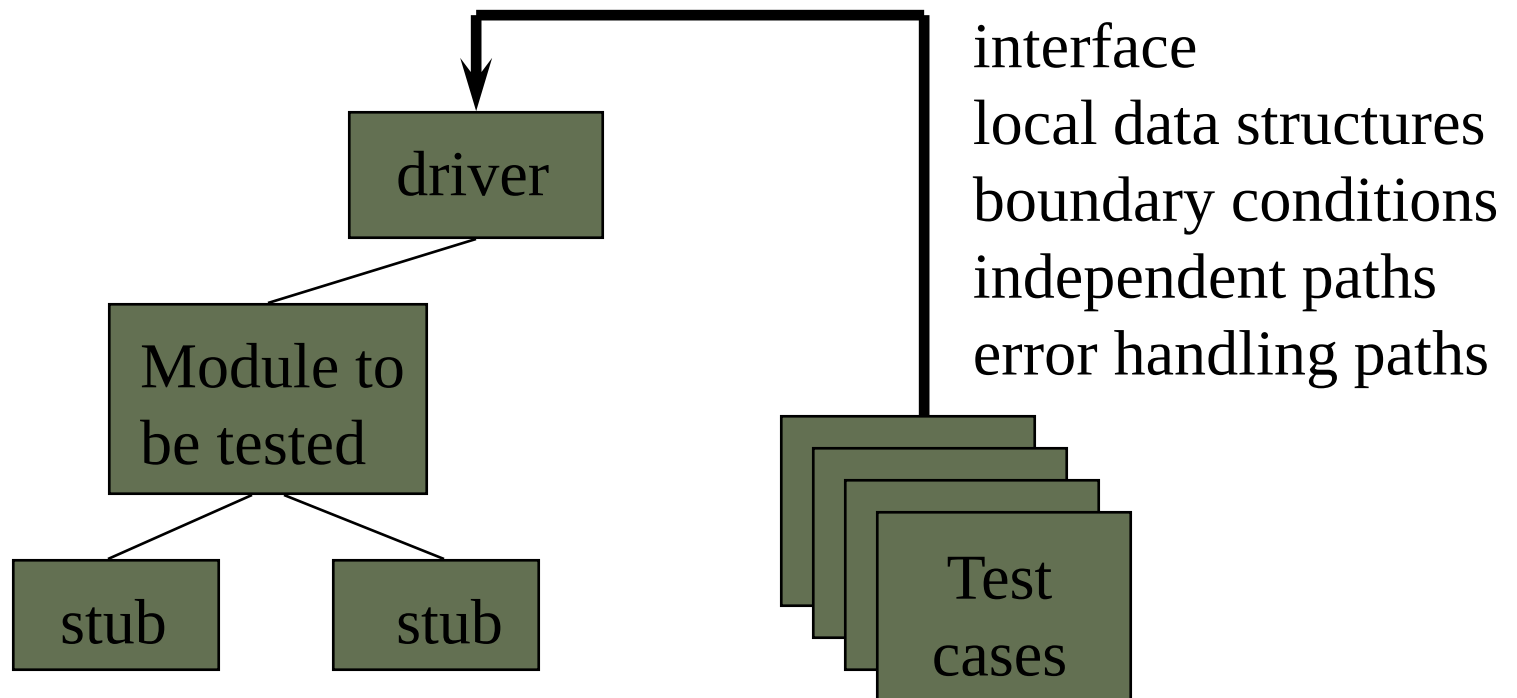
- Corresponds to class testing in the OO context.

Makes heavy use of white-box testing.

UNIT TESTING

Unit Test Generation Considerations:

Review Design information - develop unit test cases.



Unit Test Generation

Interface considerations

- # of input parameters = # arguments?
- Parameter and argument attributes match?
- Parameter and argument units match?
- Order correct (if important)?
- Number and order of arguments for built-ins?
- References to parms not associated with entry point?
- Attempt to modify input-only arguments?
- Global variable definitions consistent?
- Constraints passed as arguments?

Unit Test Generation

External I/O considerations

- Files attributes correct?
- OPEN/CLOSE correct?
Format specification matches I/O statement?
- Buffer size matches record size?
- Files opened before use?
- EOF handled correctly?
I/O errors handled?
- Textual errors in output?

Unit Test Generation

Data structure considerations

- Improper or inconsistent typing?
- Erroneous initialization or default values?
- Incorrect variable names?
- Inconsistent data types?
- Underflow, overflow and addressing exceptions?

Unit Test Generation

Test cases must cover all execution paths

Common computational errors to be checked:

- incorrect arithmetic
- mixed mode operations
- incorrect initialization
- precision inaccuracy
- incorrect symbolic representation of expression

Other tests needed

- incompatible data types in comparisons
- incorrect logical operators or precedence
- comparison problems (e.g., == on floats)
- loop problems

Unit Test Generation

Error handling tests

- Exception-handling is incorrect?
- Error description is unintelligible, insufficient or incorrect?
- Error condition causes system interrupt before error handling completed?

INTEGRATION TESTING

A systematic approach for constructing program structure while conducting tests to uncover errors associated with interfacing.

Tendency for *Non-Incremental* integration.. Big Bang approach Chaos !! (usually).

Incremental integration - program is constructed and tested in small segments.

- Top-Down Integration testing
- Bottom-Up Integration testing

INTEGRATION TESTING

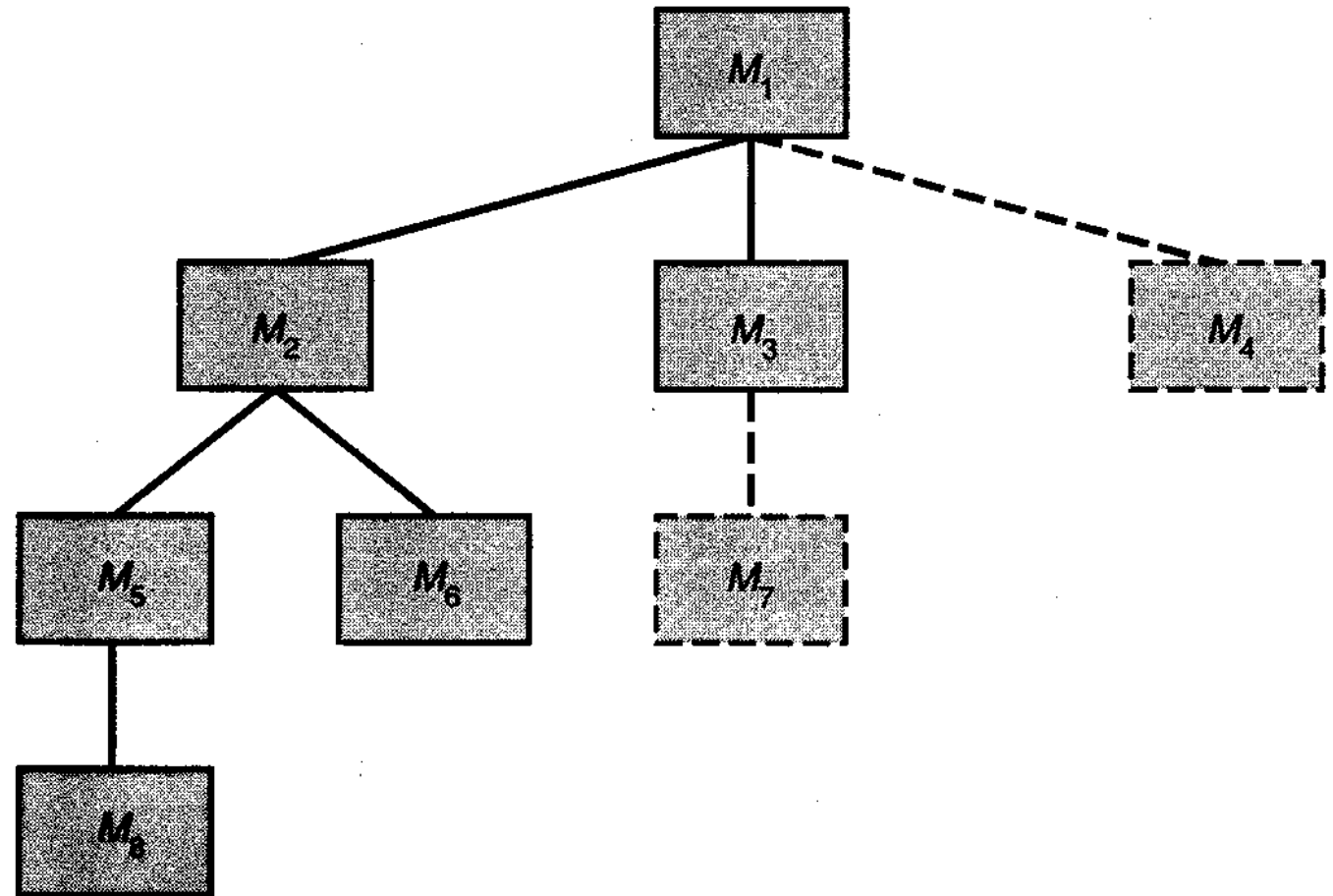


FIGURE 17.7.
Top-down integration

INTEGRATION TESTING

Top-Down Approach

Begin construction and testing with main module.

- Stubs are substituted for all subordinate modules.

Subordinate stubs are replaced one at a time by actual modules.

Tests are conducted as each module is integrated.

On completion of each set of tests, another stub is replaced with the real module.

Regression testing may be conducted to ensure that new errors have not been introduced.

INTEGRATION TESTING

Top-Down Approach :

- Advantages:
 - Verifies major control or decision points early in the test process.
 - With the use of depth-first integration testing, a complete function of the software can be demonstrated. --
Confidence builder for developer/customer.
- Disadvantages:
 - Since stubs replace lower level modules, no significant data can flow upwards to the main module.

INTEGRATION TESTING

Bottom Up Approach :

- This approach begins construction and testing with modules at the lowest levels in the program structure.
- Low-level modules are combined into clusters.
- A driver is written to coordinate test case input and output.
- The cluster is tested.
- Drivers are removed and clusters are combined moving upward in the program hierarchy.

Bottom Up Approach

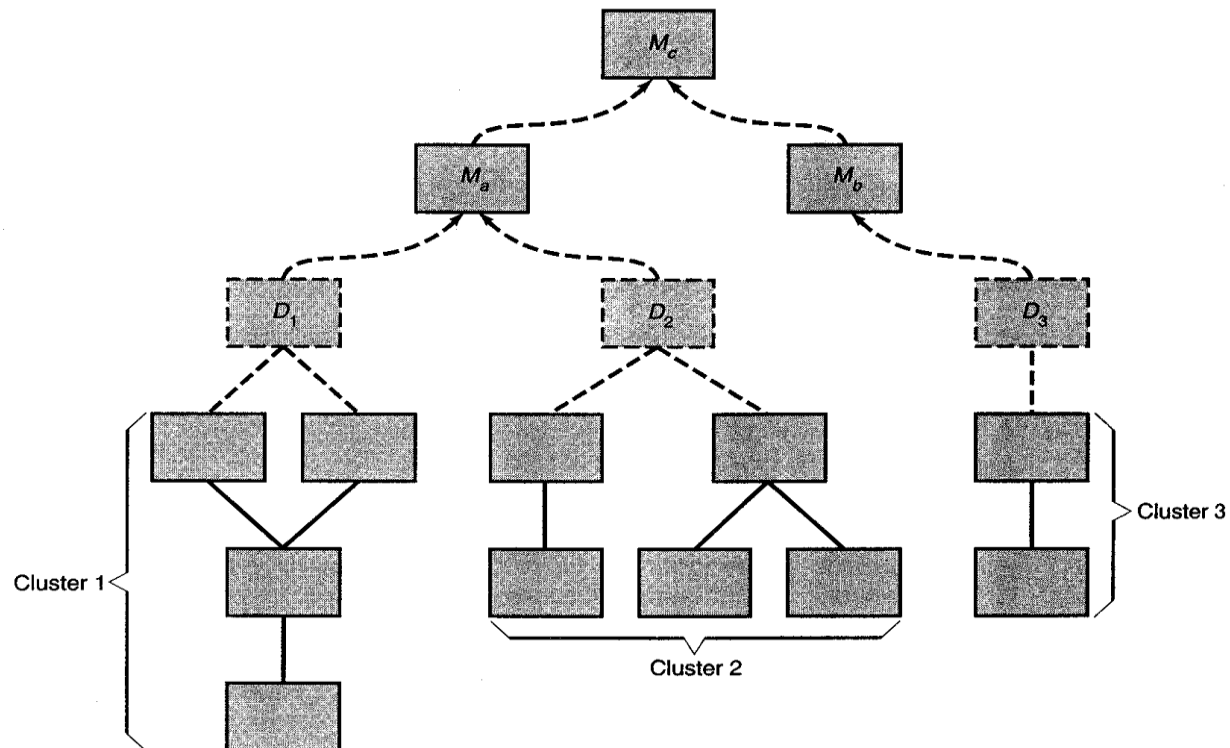


FIGURE 17.9. Bottom-up integration

INTEGRATION TESTING

Bottom Up Approach

- Advantages:
 - Easier test case design and lack of stubs.
- Disadvantages:
 - The program as an entity is does not exist until the last module is added.

Sandwich Testing:- combined approach

- Top down strategy for upper levels and Bottom up strategy for subordinate levels.

INTEGRATION TESTING

Regression Testing

- Re-execution of some subset of tests already conducted to ensure that the new changes do not have unintended side effects.

The Regression test suite should contain three different classes of test cases :

- A representative sample of tests that will exercise all software functions
- Additional tests that focus on functions that are likely to be affected by the change.
- Tests that focus on software components that have changed.

VALIDATION TESTING

It provides final assurance that software meets all functional, behavioral, and performance requirements.

- Exclusive use of Black-box testing techniques.

After each validation test case either software conforms to specs or a deviation from specs is detected and a deficiency list needs to be worked.

Alpha and Beta testing

- Alpha test -- At developer's site by customer.
- Beta test -- At customer's site in a "live" environment.

SYSTEM TESTING

A series of tests to verify that all system elements have been properly integrated.

Recovery Testing:

- Forces software to fail in a variety of ways and verifies that recovery is properly performed.

Security Testing:

- Attempts to verify the software's protection mechanisms. The software designer tries to make penetration cost more than the value of information obtained by breaking in.

SYSTEM TESTING

Stress Testing:

- Executes the system in a manner that demands resources in abnormal quantity, frequency or volume.

Performance Testing:

- To test the run time performance of a system within the context of an integrated system.

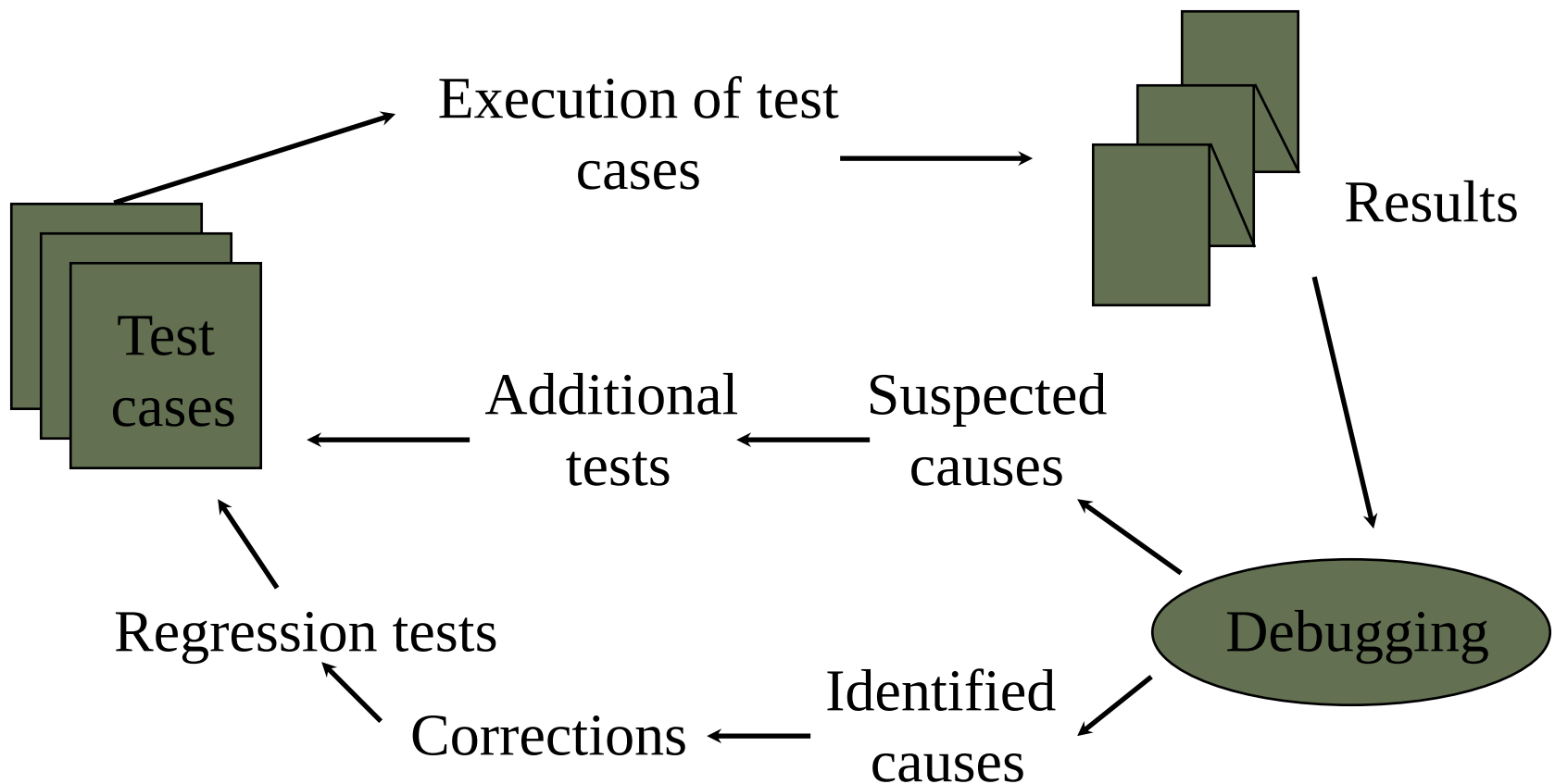
THE ART OF DEBUGGING

Debugging is a consequence of successful testing -- when a test case uncovers an error, it is the debugging process that results in the removal of the error.

Debugging is an ART. The external manifestation of the error and the cause of the error normally do not share an obvious relationships.

THE ART OF DEBUGGING

The Debugging process



THE ART OF DEBUGGING

Debugging Approaches

- Brute force : - Take memory dumps, invoke run time traces. Least efficient and quite common.
- Backtracking :- Once an error is uncovered, trace your way back to the cause of the error.
- Cause Elimination : - Isolate potential causes, devise cause hypotheses tests to isolate bug.
- Use of debugging tools